The original **FL Language Manual** you can find at:
https://softwarepreservation.computerhistory.org/FP/doc/Backus_et_al-FL_Language_Manual_Parts_1_and_2-RJ7100-1989.pdf

Alberto Paoluzzi and his PLaSM-team
https://web.archive.org/web/20200802000139/http://www.plasm.net/
https://plasmlanguage.org/apaoluzzi/

Tutorial

The design language PLaSM is a geometry-oriented extension of a subset of FL.

FL Language

FL (programming at Function Level) is an advanced language for functional programming developed by the **Functional Programming Group of IBM Research Division at Almaden** (USA) 1989 [BWW90,BWWLA89]. The FL language, on the line of the Backus' Turing lecture [Backus78] introduces an algebra over programs, i.e.~a set of algebraic identities between functional expressions, for formally reasoning about programs. This language enjoies some interesting features. In particular, programs are easily combined, so that new programs are obtained in a simple and elegant way; and one may find simpler equivalent programs, both at design and at compilation times. Great advantages are so obtained in style and efficiency of program prototyping.

PLaSM Language

PLaSM, (the Programming LAnguage for Solid Modeling) is a ``design language", developed by the CAD Group at the Universities of Roma ``La Sapienza" and ``Roma Tre" [PS92, PPV95]. The language is strongly inFLuenced by FL. With few sintactical differences, it can be considered a geometric extension of a FL subset. Among the strong points of PLaSM we cite the functional approach, which allows to compute with geometries as well with numbers and functions, and its the dimension-independent implementation of geometric data structures and algorithms. The first feature results in a very natural and powerful approach to parametric geometry. The second one, coupled with the "combinatorial engine" of the FL language, gives an amazing descriptive power when computing with geometry.

Elements of FL syntax

FL is a pure functional language based on combinatorial logic.
The language introduces an algebra over programs (a set of algebraic identities between functional expressions) for reasoning formally about programs, so that one may find simpler equivalent programs (both at design and at compile times); programs are easily combined, so that new programs are obtained in a simple and elegant way; great advantages in style and efficiency of program development are achieved.

Primitive objects
characters, numbers and truth values

Expressions
primitive objects, functions, applications and sequences

Sequences
expressions separated by commas and contained within a pair of angle brackets: <5, fun>

The application expression
exp1:exp2 applies the function resulting from the evaluation of exp1 on the argument resulting from the evaluation of exp2.

Binary functions
can also be used in infix form: +:<1,3> = 1 + 3 = 4

Application associates to left:
f:g:h = (f:g):h

Application binds stronger than composition:
f:g ~ h = (f:g) ~ h

FL combining forms and functions:
The construction combining form CONS
allows to apply a sequence of functions to an argument producing the sequence of applications:
CONS:< f1,...,fn >:x = [f1,...,fn]:x = < f1:x,...,fn:x >

E.g. CONS:<+,->, written also [+,-], when applied to the argument <3,2> gives the
sequence of applications:
[+,-]:<3,2> = <+:<3,2>,-:<3,2>> = <5,1>;

The apply-to-all combining form AA
applies a function to a sequence of arguments giving a sequence of applications:
AA:f:< x1,...,xn > = < f:x1,...,f:xn >;

The identity function ID
returns its argument unchanged: ID:x = x;

The constant function K
is evaluated, for whatever x2, as: K:x1:x2 = x1;

Binary composition ~ of functions
is defined as (f ~ g):x = f:(g:x);

N-ary composition COMP of functions
is also allowed: COMP:< f,h,g >:x = (f ~ h ~ g):x = f:(h:(g:x));

The conditional form
IF:< p,f,g >:x is evaluated as follows:
IF:< p,f,g >:x = f:x if p:x = TRUE g:x if p:x = FALSE

The insert right and insert left combining forms (INSR and INSL)
allow to apply a binary function on a sequence of arguments of any length:
INSR:f:< x1,x2,...,xn > = f:< x1, INSR:f:< x2,...,xn > >
INSL:f:< x1,...,xn-1,xn > = f:< INSL:f:< x1,...,xn-1 >, xn >,
where INSR:f:< x > = INSL:f:< x > = x

The catenate function CAT
appends any number of input sequences creating a single output sequence:
CAT:<< a,b,c >,< d,e >,...,< x,y,z >> = < a,b,c,d,e,...,x,y,z >

The distribute right and distribute left functions (DISTR, DISTL)
generate a sequence of pairs:
DISTR:<< a,b,c >, x> = << a,x >, < b,x >, < c,x >>
DISTL:< x,< a,b,c >> = << x,a >, < x,b >, < x,c >>

Append:

The important Curry function
This function embodies the fundamental idea of FL.
C:f:x:y = f:<x,y>

You can find it at:
https://web.archive.org/web/20170402054505/http://plasm.net/_Media/listings.png

Selector functions
s1, s2, s3, ...
Select the corresponding values from the sequence:
s1 : <10,20,30> = 10
s2 : <10,20,30> = 20
s3 : <10,20,30> = 30

PLaSM Approach to Geometry

PLaSM, the Programming LAnguage for Symbolic Modeling, is a functional "design language" designed in recent years by the CAD Group at the University of Rome "La Sapienza" (Italy) and currently developed by the Geometric Computing Group at the University of Roma Tre.

PLaSM features

Programming approach to "generative" modeling
Geometrical objects generated by language expressions
Algebraic calculus over embedded polyhedra
Dim-independent approach to geometry repr and algorithms
Extended variational geometry (classes of objects with varying topology and shape)
Geometry-oriented extension of the functional language FL
Validity of geometry syntactically guaranteed
Domain broader than in solid modelers (points, wire-frames, surfaces, solids, etc.)

PLaSM is a geometry-oriented extension of a subset of FL:

it is a functional "design language"
can evaluate expressions whose value is a polyhedral complex
can produce higher-level functions in the FL style
no free nesting of scopes and environments is allowed
no pattern matching is provided
allows identifiers for any language object

The syntax of PLaSM is very similar to the one of FL. The differences only concern the meaning of few characters and the use of multiple lists of formal parameters of functions. Two new combining forms AC (apply-in-composition) and AS (apply-in-sequence) are introduced, in order to use a function with a number of actual parameters greater than the number used in the function definition.

Example

SEL is a primitive operator with a specification parameter i, such that when applied to a sequence the i-th sequence element is returned:

SEL:2:<13, 4.5, ID> = 4.5

By using the combining form AC and AS, the SEL operator can work in two different ways:

AC:SEL:< 3,1 >:<< 1,3,8,7 >, 89, fun>= (SEL:3 ~ SEL:1):<< 1,3,8,7 >, 89, fun> = 8
AS:SEL:< 3,1 >:<< 1,3,8,7 >, 89, fun>= [SEL:3, SEL:1]:<< 1,3,8,7 >, 89, fun>= < fun, < 1,3,8,7 >>

PLaSM uses a case unsensitive alphabet;
Allows for partially specified (curried) functions.
Allows for overloading of operators:

3+2; 3*2
ops on numbers

(sin+cos):x; (sin*cos):x;
ops on functions

pol1+pol2; pol1*pol2;
ops on polyhedra

Predefined functions

Some non geometric predefinite functions and combinatorial forms follow

Type predicates
(IsInt, IsReal, IsSeq, IsFun, IsPol)

Comparison functions
GT, GE, LT, LE, EQ, ...

Logical functions and constants
AND, OR, NOT, TRUE, <>, ...

Combining Forms
AS, AC, ...

Mathematics functions
SIN, COS, TAN, LN, EXP, ...

Sequence functions
AL, AR, FIRST, TAIL, CAT, ...

Some more non-geometric operators:
repetitition operator allows for instancing n times any expression:
#:3:expr = < expr,expr,expr >

catenation of repeated sequences repeats and catenates sequences:
##:3:< a,b,c > = < a,b,c, a,b,c, a,b,c >

FROMTO operator generates integer sequences between two given extremes:
FromTo:<2,5> = 2..5 = <2,3,4,5>

INTSTO operator generates integer sequences with a given end:
INTSTO:5 = <1,2,3,4,5>

Function definition

Two kinds of functions are recognized in PLaSM:

global (or top-level) functions

Global functions may contain a definition of local functions between WHERE and END keywords. Global functions are allowed to contain formal parameters using a lambda-style. Formal parameters are specified together with a predicate wich is normally used to perform some type-checking at run-time.

local functions

The visibility of local functions is restricted to the scope of a global function.
Formal and actual parameters are given as follows:

Function definition:
DEF f (a::type1) = bodyF
DEF g (a1,...,an::type2) = bodyG
DEF h (a1::type1)(a2::type2) = bodyH (Currying)
DEF l (a1::type1; a2::type2) = bodyL

Function instancing:
f:x
g:< x1, ... , xn >
h:x1:x2
l:< x1, x2 >

Append:

The types in the function definitions are predicates like: IsInt, IsReal, IsSeq, IsFun, etc.
- Propositions as Types
- Dependent Types
Example at
https://web.archive.org/web/20170402054505/http://plasm.net/_Media/listings.png

Geometric functions

Elementary shape contructors (geometric primitives)
SIMPLEX, CUBOID, CYLINDER, ...

Affine transformations
T, S, R, H, MAT, ...

Hierarchical Operator (Assemblies with local coordinates)
STRUCT

Skeletons of a complex
@0, @1, @2, @3, ...

Constructor of 1D polyhedra
QUOTE

Cartesian Product of complexes
*

Intersection of extrusions
&&

Parametric mapping
MAP:VectorFunction:Domain

Regularized Booleans Operators
+, &, -, \, XOR

Quick Positioning and scaling
TOP, BOTTOM, LEFT, RIGHT, ...

For efficiency reasons, a special form for parameter specification has been implemented for some frequently used elementary geometric function. In particular, the functions MIN, MAX, SIZE, BOX and the affine transformation functions T, S, R, H can be specified both on a single value and on a sequence of values. E.g., T denotes translation, and T:3:2.5 means translation of 2.5 units on the third coordinate. Hence for a translation on more than one coordinate we can write ...

[...]

https://web.archive.org/web/20200703133348/http://www.plasm.net/docs/tutorial/plasm_approach_to_geometry.html